DCC Introduction and State of the Art





Jason Molgaard, Solidigm | Sakul Gupta, Micron

Future Technologies Initiative Workshop: Data-Centric Computing

OCP FUTURE TECHNOLOGIES SYMPOSIUM

October 14, 2026
San Jose, California

What This Group Works On

DATA AT REST

Computational Storage

Compute on or in the storage device — transform, filter and reduce data on its way to or from the media, so only results cross the bus.

DATA IN TRANSIT

DPU

Compute in the network path — packet, protocol, security and storage-network functions handled before the host ever sees them.

DATA IN USE

PIM and PNM

Compute in or beside memory — operate on the working set where it already lives instead of hauling it to an accelerator.

PAPER ONE · PUBLISHED AUG 2025

Near Data Computing: Present & Future

- What each technology is, and where each one applies
- Use cases: key-value CSD, search in storage, database, video, data integrity, OneMCC, AI framework, VectorDB offload
- Layer abstractions, programming options, and management via Redfish

PAPER TWO · IN DRAFT

Near Data Computing Framework

- How an application actually reaches an NDC device, in any of its forms
- Frameworks, APIs, data formats, runtime, and architecture options
- The gaps that have to close before NDC is ordinary infrastructure

Same thing, different names: Near Data Computing = Data Centric Computing · Computational Storage = in-storage compute = storage intelligence · PIM = in-memory processing · PNM = near-memory processing

Watch for "computational memory" — the industry uses it for both PIM and PNM, and you will hear both today.

Why Near Data Computing

THE DURABLE WHY

Data gravity, not this year's workload, is why this group exists

- Dave McCrory, 2010: moving computation is easier than moving data — and the pull strengthens as latency and throughput requirements rise
- Dennard scaling ended in the mid-2000s and Moore's law has slowed; over 92% of servers are already virtualized, so utilization has little left to give
- Domain-specific architectures are the remaining lever — compute purpose-built for the state the data is already in
- Data centers drew 240–340 TWh in 2022 — servers and storage are more than half of it, and Ireland went from 5% to 21% of metered electricity since 2015

The Climate Neutral Data Center Pact commits to a 53% emission reduction by 2030

WHAT AI ADDED

Memory capacity, bandwidth and data movement are now system resources as critical as compute

- GPU-local HBM rarely holds the working set; concurrency, long context and reused attention and KV caches consume it and cap throughput
- Memory limits bind even when GPU compute is not saturated
- HBM, NVLink and coherent CPU–GPU links reduce the cost of movement — the execution model is unchanged, and data must still arrive at the GPU
- The IEA expects electricity use from AI and data centers to double within four years

So move selected operations to where the data already resides — that is Near Data Computing

Near Data Computing: One Idea, Four Device Classes

PIM

Processing in Memory

also: in-memory processing, computational memory

Processing elements bound to memory banks, triggered by ordinary memory commands from the controller.

10–23.2× performance · 1.64–5.23× energy

PNM

Processing near Memory

also: near-memory processing, computational memory

Discrete accelerator with its own attached memory, separate from host CPU and host memory. Often CXL-attached.

1.39–1.77× performance · 31.6% energy

CS

Computational Storage

also: in-storage compute, storage intelligence

Compute on or in the storage device. SNIA defines three types: processor, drive and array.

4.3× performance · 36% energy

DPU

Data Processing Unit

also: SmartNIC, smart switch

Compute in the network path, operating on data in flight rather than data at rest or in use.

54× performance · 24–29% energy

Data at rest → Computational Storage data in transit → DPU data in use → PIM and PNM

Measured gains above are from paper one, Table 1 — one idea, four places to put it: computation where the data already resides.

The Gap Is Software, Not Silicon

TODAY'S STACK

- PyTorch and TensorFlow execution paths are designed around CUDA GPU execution models
- Mature and highly optimized — but vendor-specific and closed
- New device classes arrive through device-specific APIs, custom runtime paths and bespoke memory management

WHAT BREAKS

- Code paths and execution flows diverge per vendor and per device type
- Framework assumptions and optimizations are disrupted — performance becomes unpredictable
- For NDC, where access patterns and execution ordering are critical, GPU-centric abstractions limit effectiveness

WHY EXISTING ABSTRACTIONS FALL SHORT

- Library-based: OpenCL, oneAPI/SYCL, vendor BLAS — runtime dispatch onto pre-optimized kernels
- Compiler-based: OpenXLA, GLOW, TVM, Triton — graph analysis, fusion, code generation
- In practice: unmaintained, narrowly applicable, or demanding deep compiler and runtime expertise

Enabling NDC is a framework integration problem — memory management, dataflow assumptions and execution ordering — not simply a matter of providing compute capability.

Anatomy of the NDC Framework

Applications & AI frameworks
PyTorch · TensorFlow · JAX · Hugging Face · ExecuTorch · FAISS

Compilers & runtime
OpenXLA · Triton · IREE · GLOW · optimized libraries

APIs — control plane & data plane
SNIA CS API · NDC API · OpenCL · MPI collectives · SDXI · allocation & pooling

Data formats & transforms
Apache Arrow · ONNX · HDF5 · Substrait

NDC devices
PIM · PNM · Computational Storage · DPU

WHAT THE FRAMEWORK MUST DEFINE

- Access model — load/store, queued commands, or both
- Addressing of data source and destination — memory, storage, in flight
- Shared data model, address transformation, coherency
- Identification and parameters of compute operations
- Data format and layout understood on both sides
- Separation of control plane from data plane, consistent with Redfish and CXL FM API

Open Formats Are the Contract

FORMATS FOR DATA

Apache Arrow — columnar layout with relocatable, offset-based addressing: zero-copy sharing across processes, languages and address spaces

HDF5 — self-describing objects and relative addressing; chunked storage gives a natural unit of work, and the filter pipeline, VFD and VOL layers are ready attach points for computation

ONNX — model interchange into device backends and compilers

No padding, no re-alignment copies, no serialization tax when data crosses a runtime boundary

FORMATS FOR COMPUTATION

Substrait and Arrow compute functions — let frameworks, language runtimes and accelerator runtimes share function semantics declaratively

The alternative — shipping code and trans-compiling it across instruction sets — has held back NDP adoption and limited what can be offloaded

Ahead: declared computation is data lineage, enabling recompute and lazy evaluation to save capacity in the most oversubscribed tiers — GPU HBM, activations, KV cache

An accelerator can only act on data it can navigate, and on a function it can name. **Open formats supply both.**

APIs: Generalize a Ratified Standard

SNIA CS API v1.1 (ratified May 2025)

- Transport agnostic across PCIe, Ethernet, NVMe-oF and CXL; 30+ functions; layered resource model of engine, environment, function and device memory
- Three completion models, plus batch requests with serial, parallel and hybrid dependencies
- Plugin framework maps abstract functions onto a device's own interface — with host-CPU fallback when no device is present

The sequence it encodes is not storage-specific

Discover → activate environment and function → allocate device memory → move operands in → queue compute → take completion → move results out

PNM is the closest structural fit; **DPU** fits but needs a stream as source and sink; **PIM** is hardest — the plugin turns a queued request into a kernel launch, at whole-operator granularity such as GEMV

WHAT HAS TO CHANGE TO COVER EVERY NDC CLASS

Device class
A class field in device properties, so applications can discover CSD, CSP, PIM, PNM or DPU

Discovery
Resolve a device from a memory region, NUMA node, CXL handle or network flow — not only a file path

Source & sink
One descriptor-based abstraction covering media, host memory, device memory, shared regions and streams

Memory model
Attributes for host mappability, coherency and address space sharing on allocated device memory

Operands & registry
Data type, shape, stride and layout descriptors, plus agreed names and signatures for well-known functions

Placement
Topology and affinity hints so a runtime can decide where function and data belong

Two Architectures, Two Markets

DEVICE FIRST

Memory is exposed for direct host access; the accelerator sits behind it

- Command driven over an existing transport such as NVMe; commands may terminate at the controller and reissue internally
- Load/store host access to memory with peer-to-peer access to built-in storage; operators enumerated and installed
- Fits line-rate, transparent, cost-sensitive functions — often hardened in an FPGA, configured out of band

Examples: NVMe computational storage devices, Samsung SmartSSD, ScaleFlux compressed SSD

ACCELERATOR FIRST

Memory sits behind the accelerator; the host talks to the accelerator directly

- Storage becomes a tier behind accelerator memory; pipelined operation over memory-to-memory transports such as CXL and UALink
- Shared virtual address space between host and accelerator; coherence adopted or replaced by low-coordination sharing
- Soft-programmed functions where function and data co-placement is the optimization — machine learning and AI workloads

Every accelerator carries a custom interface today; a standard one is an open item for this group

A third, hybrid architecture is on the table — **choosing between them is a decision the framework must survive.**

State of the Art: What Works, What Is Open

RUNNING TODAY

- **NDC API reference implementation (Contributed by Samsung)** — host-side abstraction for discovery, memory management and function execution, targeting CXL.mem devices; published in the OCP GitHub organization
- **LLaMA 3.2-1B GEMV offload** — ExecuTorch CPU BLAS path routed to a PNM device; the matrix stays resident and only the smaller vectors cross the interconnect
- **FAISS IVFFlat L2-kNN** — RAG vector search executed where the index resides, returning results instead of candidate vectors
- **Plugin-based reuse** — a PIM, PNM or DPU vendor can map today's CS API onto OneMCC, DOCA or ROCm without waiting for a revised standard
- **Formats in production** — HSDS, the DAOS VOL connector and PDC already evaluate selections and run analysis server-side over HDF5

OPEN FOR THIS WORKSHOP

- Coherent shared address space, non-coherent, or both — and what a good API for either would measure
- Naming data in flight: a stream as the source or sink of a compute request
- A registry of well-known function names and signatures, so portability survives a change of vendor
- Function and data placement — topology, affinity, and who decides: compiler, runtime or application
- Device-managed operation, beyond today's host-orchestrated interface
- Partitioning and open object formats at cluster scale, where one framework meets a million GPUs
- How the control plane aligns with Redfish and the CXL fabric manager API
- Security — every NDC device adds an attack surface the framework has to account for

LLM Serving Is Becoming a Data-Movement Problem

As models scale, bottlenecks increasingly emerge outside the GPU

KV CACHE

Capacity grows with concurrency and context

MEMORY BANDWIDTH

Tokens wait on data delivery

FABRIC CONGESTION

PCIe and accelerator links saturate

MULTI-TENANCY

Scheduling must balance latency and utilization

FRAGMENTATION

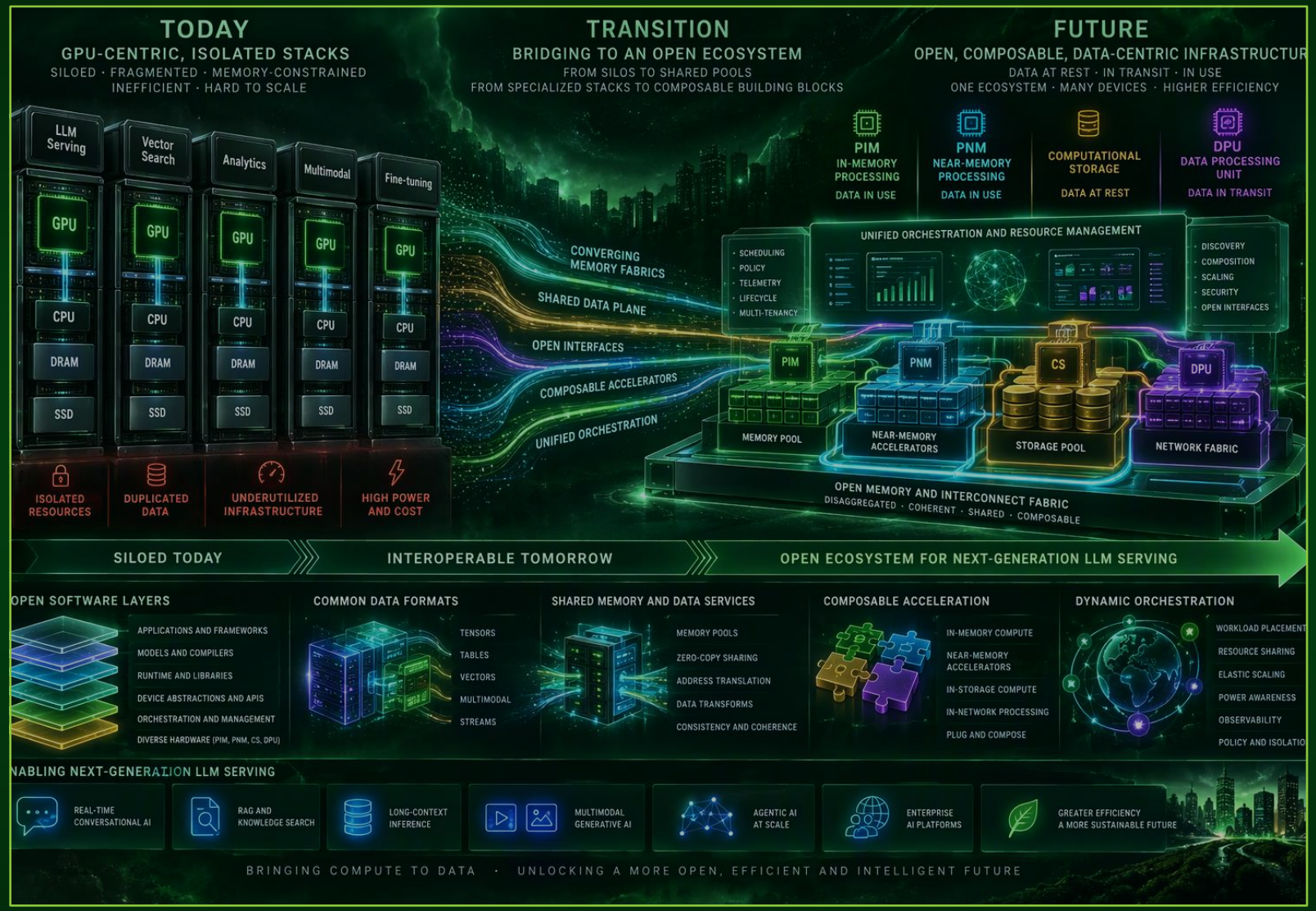
Stranded memory and compute reduce efficiency

POWER + TCO

Movement raises energy and operational cost

DCC response: place selected computation near memory, storage, and data in flight, then orchestrate resources as one system.

A Data-Centric Architecture for LLM Serving



GPU CLUSTER

Dense matrix compute

CXL MEMORY POOLS

Elastic capacity + shared KV cache

PNM / PIM

Vector search, embeddings, reductions

DPU + COMPUTATIONAL STORAGE

Data movement, retrieval, preprocessing

ORCHESTRATOR: place work where data resides

Roadmap: From Point Offloads to an Open LLM Serving Ecosystem

NOW

Prove value

- GEMV and vector-search PoCs
- Shared KV-cache experiments
- Device-specific plugins and APIs

NEXT

Converge interfaces

- Common source / sink descriptors
- Function registry and open formats
- Topology, affinity and placement hints

SCALE

Orchestrate the system

- Composable memory and accelerators
- Dynamic multi-tenant scheduling
- Security, telemetry and lifecycle control

OPEN ECOSYSTEM OUTCOMES

Lower TTFT	Higher token throughput	Better GPU utilization	Memory efficiency	Lower TCO	Sustainability

Success means portable software and measurable system outcomes, without prescribing one vendor architecture.

What You Will Hear Today (1 of 2)

FABRIC & DATA SERVICES

Bridging Compute and Memory Fabric: Data Services for Direct XPU Access — Marvell · **12:50**

Panel — Deep dive into what allows 1 million GPUs talk to thousands of smart devices — Ohio State, LANL, NVIDIA, Berkeley, Stanford · **13:50**

Where devices attach, how they are addressed, and how data is served to the XPU — the access and addressing model of the framework

AI SERVING & MEMORY

Agentic AI Serving System with Computational Shared Memory Pool — SK hynix · **13:10**

A System Architecture using Computational Memory for Differentially Private Training — SK hynix, Penn State University · **15:15**

NDC inside the serving and training path — shared memory pools, KV and context reuse, and what near-memory execution enables beyond performance

Listen for: how each speaker answers the framework questions — which access model they assume, how they name data and functions, and what they had to build because no standard existed.

What You Will Hear Today (2 of 2)

FRAMEWORKS & APIs

Towards an Extensible Framework for Near-Data Processing in AI: Integration Experiences with oneAPI, OpenXLA, and PyTorch — Samsung · **13:30**

Panel — Frameworks and APIs Alignment for cross-vendor: direction alignment and gaps, path forward toward a converged API — Marvell, PyTorch Foundation, Samsung, XCENA, SK hynix · **16:35**

PROGRAMMABILITY

Bringing General-Purpose Parallel Programming to Near-Data Processing — XCENA · **15:35**

CONTEXT & OPEN FORMATS

Near Data Processing opportunities for context engineering — Arm · **15:55**

Open Object Formats for Near Data Processing — Elephance Memory · **16:15**

Every talk today fills in a layer of this framework. **The NDC Framework white paper is where it gets written down — join the OCP Near Data Computing project and contribute.**

OCP FUTURE TECHNOLOGIES SYMPOSIUM

October 14, 2026
San Jose, California

